

# TicketFlow

A 6-Phase, Production-Style Event Ticketing Platform — .NET Architect Portfolio Project

[github.com/sivatheprogrammer/ticketflow](https://github.com/sivatheprogrammer/ticketflow) | 17 Architecture Decision Records | Tagged releases v0.1 → v0.6

TicketFlow is a self-directed portfolio project built to demonstrate .NET Architect-level skills: not just a working application, but a deliberate, incremental build across six phases — each introducing one major architectural concern, each backed by a written Architecture Decision Record justifying the choice made. The project spans clean architecture, OAuth, containerization, cloud infrastructure-as-code, service extraction, and two independent container-orchestration deployment paths on Azure.

## Technology Stack

|                |                                                                                  |
|----------------|----------------------------------------------------------------------------------|
| Backend        | ASP.NET Core 10, Clean Architecture, CQRS (MediatR), EF Core 10                  |
| Frontend       | Angular 20 LTS, standalone components, signals                                   |
| Data / Caching | SQL Server, Azure SQL, Redis (cache-aside + distributed locking)                 |
| Messaging      | Azure Service Bus (choreography-based Saga pattern)                              |
| Identity       | Microsoft Entra ID (OAuth 2.0 / OIDC, PKCE)                                      |
| Infrastructure | Terraform (IaC), Docker, Azure Container Registry                                |
| Deployment     | Azure Container Apps (primary) + Azure Kubernetes Service / Helm (demonstration) |

## The Six Phases

| # | Phase                      | Core Skill Demonstrated                                           |
|---|----------------------------|-------------------------------------------------------------------|
| 1 | Monolith Foundation        | Clean Architecture, CQRS, EF Core, Angular signals                |
| 2 | Authentication             | OAuth 2.0 / OIDC via Microsoft Entra ID                           |
| 3 | Containerization + Caching | Docker, docker-compose, Redis distributed locking                 |
| 4 | Cloud Deployment + IaC     | Terraform, Azure App Service, Azure SQL                           |
| 5 | Service Extraction         | DDD boundaries, YARP Gateway, Service Bus, Saga pattern           |
| 6 | Container Orchestration    | Azure Container Apps + AKS/Helm, cloud networking troubleshooting |

## Notable Engineering Highlights

- **Modular monolith with deliberate service extraction** — extracted an Identity service with its own database and its own EF Core migration history, wired through a YARP API Gateway, rather than defaulting to either a monolith or full microservices without justification.
- **Event-driven architecture** — choreography-based Saga pattern across Azure Service Bus queues for the booking confirmation flow, with no central orchestrator.

- **Infrastructure as Code across two deployment models** — the same Terraform-provisioned Azure resources (SQL, Service Bus, ACR) feed both an Azure Container Apps deployment and a full AKS + Helm deployment, built specifically to compare operational trade-offs firsthand rather than in theory.
- **Real production-style debugging, not staged demos** — 17 ADRs, 8 of them documenting actual incidents hit while building the AKS track (VM quota mismatches, Kubernetes Secret naming collisions, a .NET configuration binder silently falling back to a wrong default, and an Azure Load Balancer health probe silently dropping all external traffic) — each with a clear symptom → root cause → fix narrative.

## Signature Debugging Story

**The Azure Load Balancer health probe issue:** after fully deploying TicketFlow's services to AKS, the application worked flawlessly when tested from inside the cluster — but was completely unreachable from the public internet, across two independent networks. Root cause: Azure's Load Balancer defaults its health probe to GET /, which the Gateway doesn't serve (only /api/\* is routed) — so Azure silently marked a fully healthy backend as unhealthy and dropped all external traffic, with no trace of the failed probes in any application log. Fixed by redirecting the health probe to nginx-ingress's dedicated /healthz endpoint via a Kubernetes Service annotation. A clean example of diagnosing a failure at the boundary between application and cloud infrastructure, where the symptom (external timeout) and the cause (a mismatched health-check path) live in entirely different layers of the stack.

## Skills at a Glance

|                            |                            |                               |
|----------------------------|----------------------------|-------------------------------|
| Clean / Onion Architecture | CQRS & MediatR             | Domain-Driven Design          |
| OAuth 2.0 / OIDC           | Docker & docker-compose    | Redis (caching & locking)     |
| Terraform / IaC            | Azure Container Apps       | Kubernetes (AKS) & Helm       |
| API Gateway (YARP)         | Event-driven / Service Bus | Saga pattern                  |
| Cloud networking debugging | Git PR-based workflow      | Architecture Decision Records |

---

Full technical write-up, all 17 ADRs, and architecture diagrams available at [github.com/sivatheprogrammer/ticketflow](https://github.com/sivatheprogrammer/ticketflow)